

AI4Good 2.0 Hackathon

AWS Services — Setup & Quick-Start Guide

Everything you need to go from account login to a running prototype.

Prepared by Blend360 • Audience: Hackathon participants

1. Initial account setup

Before touching any service, get the account into a known-good state: sign in, rotate the temporary password, and protect the user with MFA. Five minutes here saves an hour of pain.

1.1 Sign in and change your password

Your team has been issued an IAM user inside a dedicated AWS account. The credentials look like this:

```
Account ID: 209556026518
Sign-in URL: https://signin.aws.amazon.com/console
Username:    Hackathon-Team-02
Temp password: *****
```

Steps:

https://signin.aws.amazon.com/oauth?redirect_uri=https%3A%2F%2Fconsole.aws.amazon.com%2Fconsole%2Fhome%3Fca_oauth_flow_id%3D182%26hashArg%3D%2523%26authcode...

Provide feedback Multi-session disabled English

aws

IAM user sign in

Account ID or alias (Don't have?)
616463666983

☐ Remember this account

IAM username
Hackathon-Team-01

Password
[masked]

☐ Show Password [Having trouble?](#)

Sign in

[Sign in using root user email](#)

[Create a new AWS account](#)

Search foundation for agentic AI
Build agents with real-time retrieval and instant scaling
[Learn more](#)

By continuing, you agree to [AWS Customer Agreement](#) or other agreement for AWS services, and the [Privacy Notice](#). This site uses essential cookies. See our [Cookie Notice](#) for more information.

https://aws.amazon.com/opensearch-service/features/serverless/fsc_icampaig... © 2025 Amazon Web Services, Inc. or its affiliates. All rights reserved.

4. You'll be prompted to change the password, after which confirm changes. Old password is your temp password provided by the team and new password rules are :

- The password must be at least **14 characters long**
- It should include **symbols** (such as @, #, \$, etc.)
- It must contain **numbers**
- It should include **uppercase letters** (A–Z)
- It should also include **lowercase letters** (a–z)

https://signin.aws.amazon.com/console?action=changePassword&userType=individual&url=https%3A%2F%2Fconsole.aws.amazon.com%2Fconsole%2Fhome%3Fca_oauth_flow_id%3D182...

Provide feedback English

aws

Password reset

Your account (616463666983) password has expired or requires a reset.
To continue, please verify your old and set a new password for **Hackathon-Team-01** (not you?).

Old Password
[masked]

☐ Show Password

New Password
[masked]

Confirm New Password
[masked]

☐ Show Password

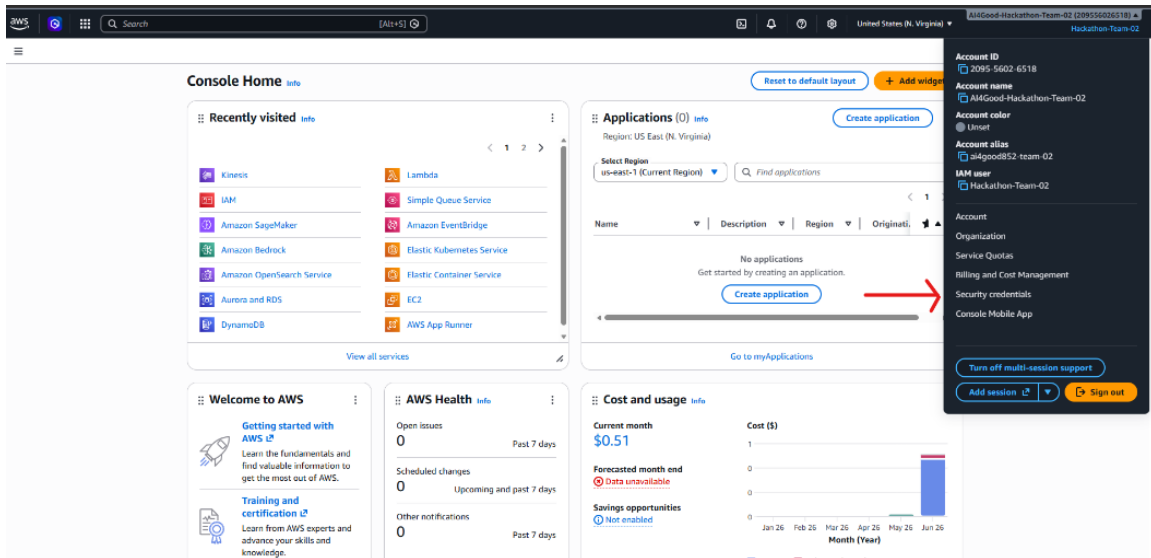
Confirm Password Change

[Sign in to a different account](#)

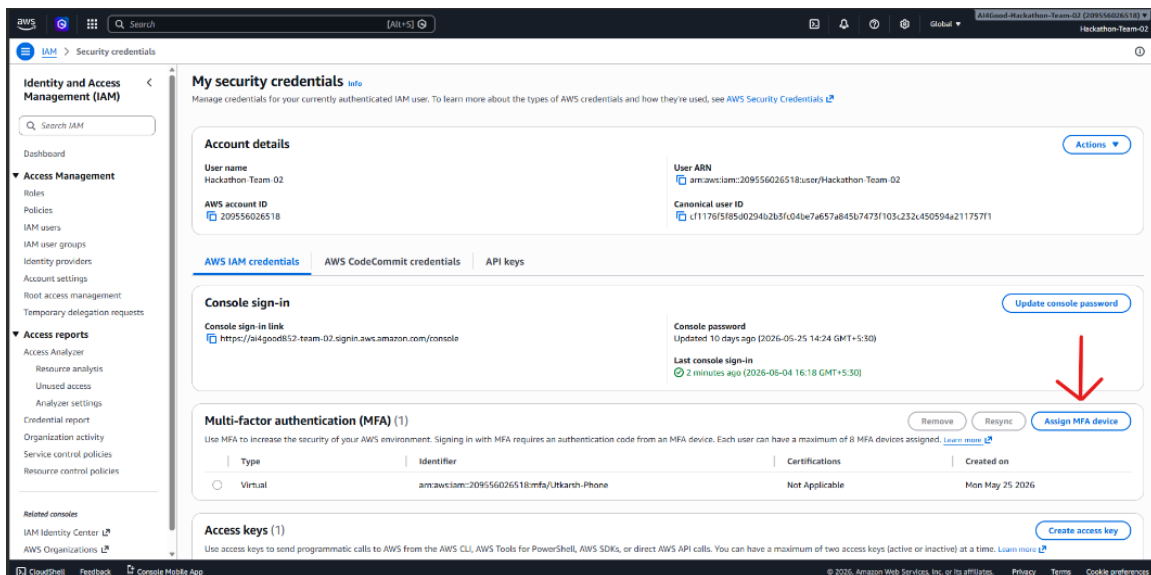
1.2 Enable Multi-Factor Authentication (MFA)

Every participant **MUST** enable MFA and logout and then re – login to the account with new password and MFA. **Without MFA, no services will be accessible to the participants.**

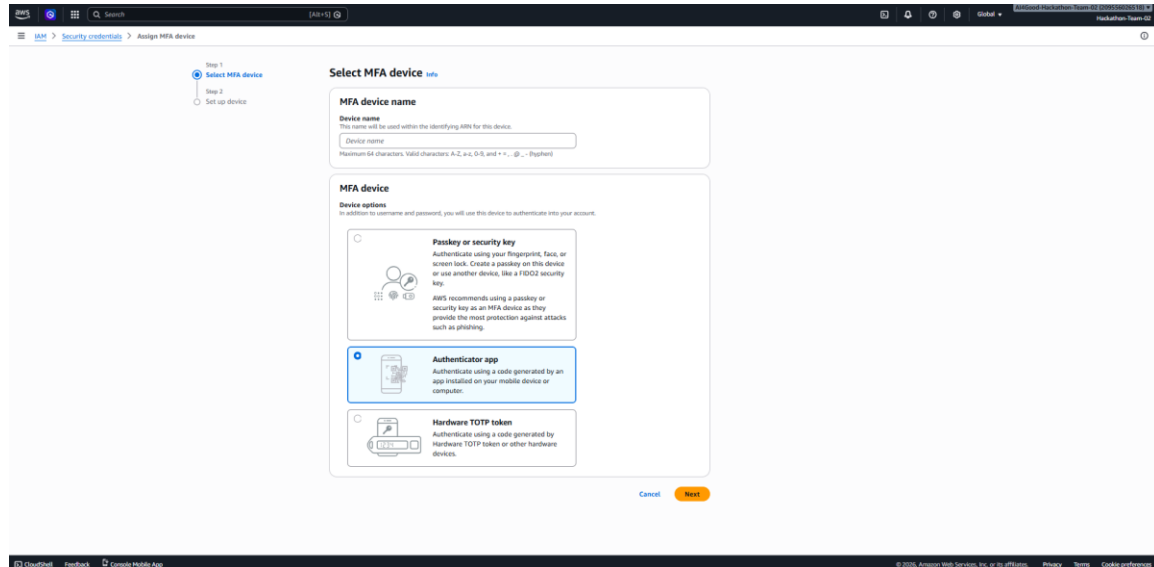
1. Download **Google/Microsoft Authenticator App** on your mobile devices.
2. Top-right of the console, click your username → Security credentials.



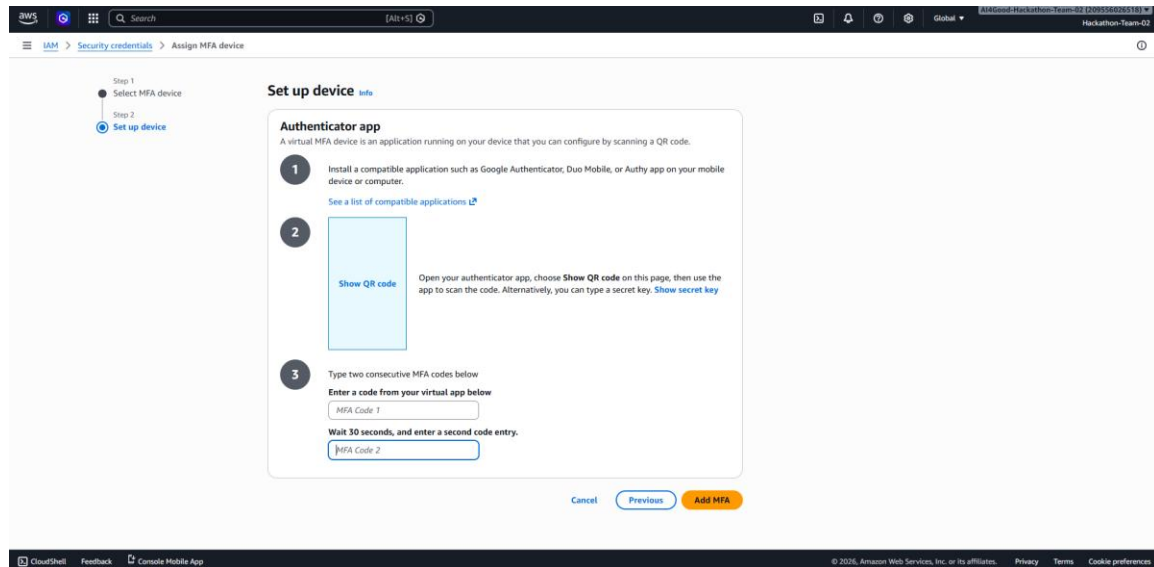
3. Scroll to Multi-factor authentication (MFA) and click Assign MFA device.



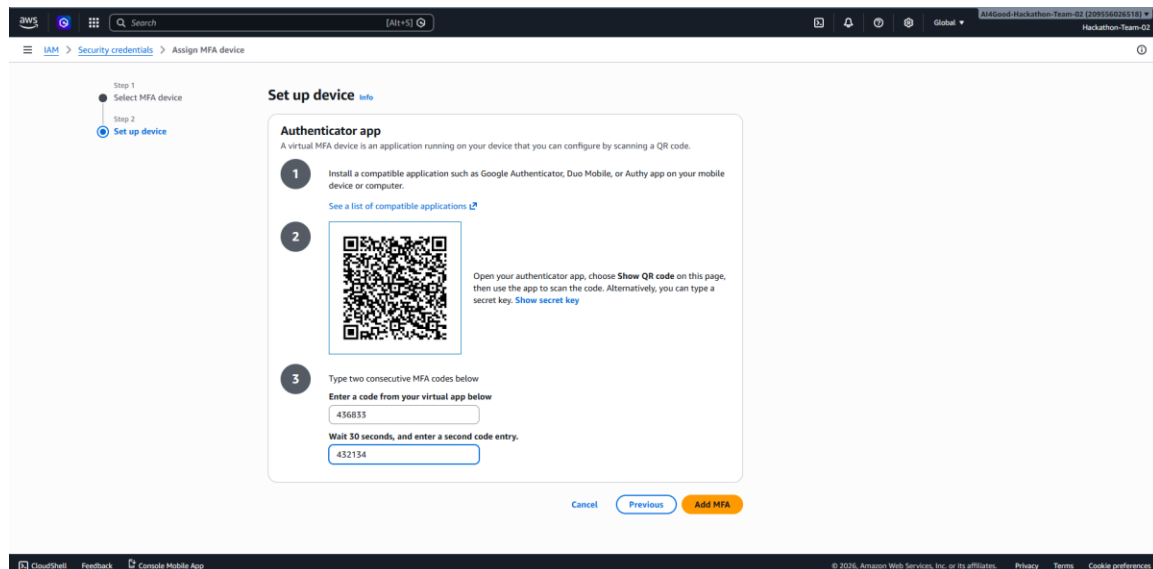
- Pick Authenticator app option, assign device name it (e.g. team-02-mfa), and click Next.



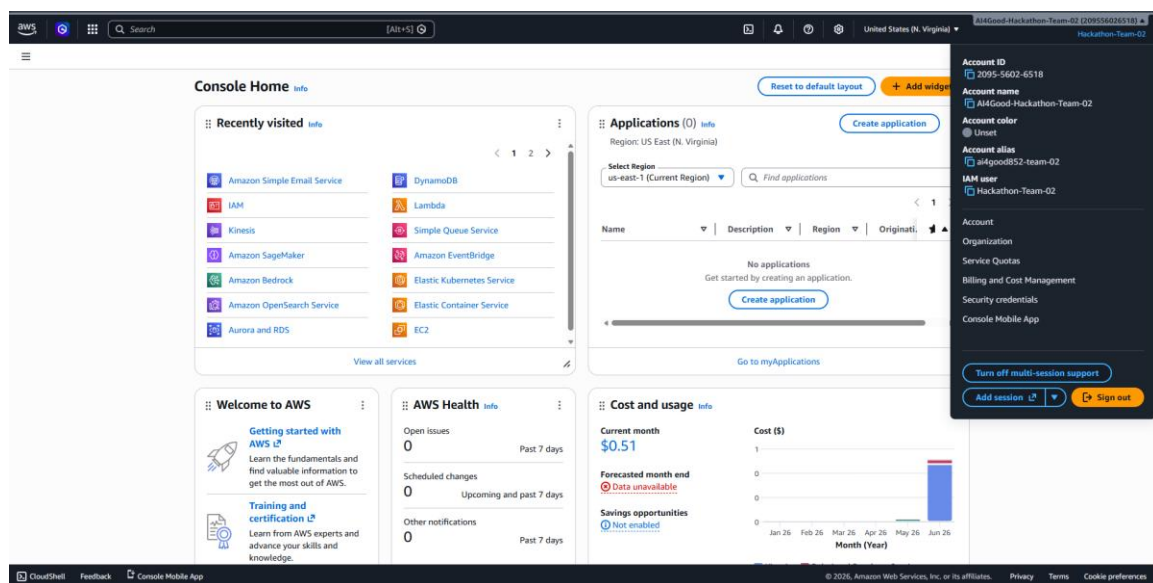
- Click on Scan QR code and Open **Google Authenticator/Microsoft Authenticator** on your phone. Scan the QR code shown in the AWS console.



- Enter two consecutive 6-digit codes from your app and click Add MFA.



7. Once the MFA is done, Logout and sign-in again to use the services.



[Tutorial: Setup MFA on AWS account — IAM Multi-factor Authentication](#)

1.3 Set your region to us-east-1

All hackathon services are provisioned in us-east-1 (N. Virginia). Use the region picker top-right of every console page and confirm us-east-1 before you create anything.

Why us-east-1: it is the oldest region, has the broadest service availability (Bedrock model access in particular), and is the default region.

2. Services overview

Below is the menu of suggested AWS services. Pick what your idea needs as most prototypes ship using only four or five. You can pick other AWS services outside of this list according to your project needs.

Category	Services in this guide
Frontend & Hosting	Amazon S3 (static hosting), CloudFront, AWS Amplify
Authentication	Amazon Cognito
Compute & APIs	EC2, AWS Lambda, API Gateway, AWS App Runner, ECS / Fargate, EKS
Messaging & Events	Amazon SQS, Amazon SNS, Amazon EventBridge
Streaming	Amazon Kinesis (Data Streams + Firehose)
Databases	Amazon RDS / Aurora, Amazon DynamoDB
Storage & Analytics	Amazon S3 (data lake), AWS Glue, Amazon Athena, Amazon Redshift, Amazon OpenSearch
Orchestration	AWS Step Functions, Amazon MWAA (Managed Airflow)
Ops & Security	CloudWatch, AWS X-Ray, Secrets Manager, AWS KMS, AWS Systems Manager (SSM), AWS WAF, AWS Shield
AI / ML	Amazon Bedrock, Amazon SageMaker, SageMaker Feature Store, Amazon Lex
Comms & CI/CD	Amazon SES, AWS CodePipeline / CodeBuild / CodeCommit

3. Frontend & Hosting

If your team is shipping a web demo, you have three good options. Pick by the shape of what you're building: a single static page (S3 + CloudFront), a single-page app or Next.js site that needs CI from a Git repo (Amplify), or a static export from a tool like Vite or Create React App (S3 + CloudFront again).

| 3.1 Amazon S3 (static website hosting)

Object storage. Cheapest way to put HTML/JS/CSS on the internet. Pair with CloudFront for HTTPS, custom domain, and a global cache.

When to reach for it: your frontend is fully static — HTML files, a built React/Vue/Svelte bundle, or a portfolio site. Costs are pennies for typical hackathon traffic.

Console quick-start:

1. S3 → Create bucket. Name it something unique like team02-demo-site. Uncheck Block all public access (you want the world to read it). Tick the acknowledgement and create.
2. Open the bucket → Properties → Static website hosting → Edit → Enable. Index document: index.html. Error document: index.html (works for SPA fallback). Save.
3. Permissions → Bucket policy → paste a policy that allows s3:GetObject on every object (template provided in AWS docs).
4. Upload your built site files (drag the dist/ or build/ folder contents into the bucket).
5. Properties → scroll back to Static website hosting → copy the Bucket website endpoint URL. That's your live site.

[Tutorial: Host a static website on S3 \(AWS official walkthrough\)](#)

| 3.2 Amazon CloudFront

Global CDN. Sits in front of S3, an API Gateway, or any HTTPS origin and caches it at edge locations worldwide. Free tier is generous (1 TB out, 10M requests).

When to reach for it: you put a static site on S3 and want HTTPS + a custom domain, or you have a public API that needs caching.

Console quick-start:

1. CloudFront → Create distribution.

2. Origin domain: pick your S3 bucket from the dropdown (use the S3 bucket origin, not the website endpoint — CloudFront handles the right access).
3. Origin access: Origin access control settings (recommended). Create new OAC, attach. CloudFront will print a bucket policy snippet — copy it.
4. Default cache behavior: Viewer protocol policy → Redirect HTTP to HTTPS.
5. Web Application Firewall: Do not enable for hackathon use (extra cost).
6. Default root object: index.html. Create distribution.
7. Copy the bucket policy CloudFront generated and paste it into your S3 bucket's Permissions → Bucket policy.
8. Wait 5-10 minutes for the distribution to deploy. The `d1111111abcdef8.cloudfront.net` URL is your live HTTPS site.

[Tutorial: CloudFront with S3 — secure static site \(AWS docs\)](#)

3.3 AWS Amplify Hosting

Vercel/Netlify equivalent on AWS. Connect a Git repo, Amplify builds and deploys on every push. Handles SSR (Next.js), custom domains, branch deploys, and previews.

When to reach for it: you have a React/Next.js/Vue app in GitHub and want CI/CD without writing pipelines. Strongly preferred over S3+CloudFront if your team already uses Git.

Console quick-start:

1. Amplify → Deploy an app → GitHub (or your provider). Authorize Amplify to read your repo.
2. Pick the repo and branch (usually main).
3. Amplify auto-detects the framework and proposes a build spec. For Vite/React, accept the default. For Next.js, ensure the framework dropdown shows Next.js.
4. Click Save and deploy. First build takes 3-5 minutes.
5. Once green, click the generated URL (`https://main.d1xyz.amplifyapp.com`). Subsequent commits to the branch trigger automatic rebuilds.

[Tutorial: Deploy a React app to Amplify Hosting \(AWS docs\)](#)

4. Authentication — Amazon Cognito

If your demo needs users to sign in, Cognito is the managed identity service. It handles sign-up, sign-in, password reset, and federation with Google/Apple/Facebook out of the box, so you don't write any of that yourself.

4.1 Amazon Cognito User Pools

A user directory that gives you hosted sign-in pages and JWT tokens you can validate from your API. Generous free tier (50,000 monthly active users).

When to reach for it: any app where users have accounts. Skip it if your demo is fully anonymous or if there's no API to protect.

Console quick-start:

1. Cognito → User pools → Create user pool.
2. Sign-in options: tick Email (simplest for a hackathon). Click Next.
3. Password policy: defaults are fine. MFA: No MFA (turn it on later if needed). Account recovery: Email only. Next.
4. Self-registration: enabled. Required attributes: email. Next.
5. Email provider: Send email with Cognito (free, rate-limited to 50/day — enough for a hackathon). Next.
6. User pool name: team02-users. App client: create one named team02-web, leave Generate client secret unchecked (web apps cannot store secrets). Next, then Create.
7. Open the pool → App integration tab → Domain → set a Cognito domain prefix (team02-auth). This gives you the hosted UI URL.

Cost: free for the first 50k MAU. You will not exceed this.

[Tutorial: Getting started with Amazon Cognito user pools \(AWS docs\)](#)

5. Compute & APIs

The backend tier. Pick by how your code is shaped. Lambda for event-driven functions and lightweight HTTP endpoints. App Runner if you have a Docker image and want zero-config deploys. ECS Fargate if you have multiple containers that talk to each other. EKS only if your team genuinely needs Kubernetes — for a hackathon it's almost always overkill. EC2 if you need a long-lived box you SSH into.

5.1 AWS Lambda

Run code on demand without provisioning servers. Pay only for invocations and execution time. 1M requests + 400,000 GB-seconds free every month.

When to reach for it: short HTTP handlers behind API Gateway, scheduled jobs (EventBridge cron), file-processing triggers (S3 upload → Lambda), Cognito triggers, anywhere code runs for under 15 minutes per invocation.

Console quick-start:

1. Lambda → Create function → Author from scratch.
2. Function name: team02-hello. Runtime: Python 3.12 (or Node.js 20.x).
Architecture: arm64 (cheaper, faster cold starts).
3. Permissions: Create new role with basic Lambda permissions. Click Create function.
4. In the inline code editor, replace the handler with your code. Click Deploy.
5. Click Test, give the test event a name, accept the default payload, Save, then Test. Output appears in the Execution results tab.

Recommended: 256 MB memory is a good default. Lambda's CPU scales with memory, so 128 MB is often slower per dollar than 512 MB for compute-heavy work. Set timeout to 30 seconds for hackathon experimentation.

[Tutorial: Create your first AWS Lambda function \(YouTube, updated 2024\)](#)

5.2 Amazon API Gateway (HTTP API)

Public HTTPS endpoint in front of Lambda or any backend. Use the HTTP API flavor, not the older REST API — cheaper, lower latency, easier to set up.

When to reach for it: your frontend needs to call your Lambda functions. API Gateway gives you a stable `https://abc123.execute-api.us-east-1.amazonaws.com` URL with CORS, JWT auth (via Cognito), and routing.

Console quick-start:

1. API Gateway → Build under HTTP API.
2. Add integration: Lambda → pick your function (e.g. team02-hello). API name: team02-api. Next.
3. Configure routes: Method GET, Resource path /hello, Integration target your Lambda. Next.
4. Stages: leave default (\$default, Auto-deploy on). Next, then Create.

5. Copy the Invoke URL. Append your route — `https://abc123.execute-api.us-east-1.amazonaws.com/hello` — and open it in a browser.
6. If the frontend will call this from a different origin, go to CORS and add your CloudFront/Amplify URL to Access-Control-Allow-Origin.

[Tutorial: Build an HTTP API with API Gateway and Lambda \(AWS docs\)](#)

5.3 AWS App Runner

Push a Docker image (or connect a Git repo); App Runner builds, deploys, and serves it on HTTPS. Auto-scales from zero. Closest thing AWS has to Heroku.

When to reach for it: you have a containerized web service (FastAPI, Flask, Express, Rails) and want it live with one click. Strongly preferred over Fargate when you only have one container.

Console quick-start:

1. App Runner → Create service.
2. Source: Container registry → Amazon ECR (push your image with docker push first) or Source code repository (GitHub) for direct-from-source builds.
3. Deployment trigger: Automatic on push to main.
4. Service settings: name team02-api, virtual CPU 1, memory 2 GB, port the port your app listens on.
5. Health check: HTTP, path /health (App Runner pings this — make sure your app responds 200).
6. Create & deploy. First deploy takes 3-5 minutes. Copy the .awsapprunner.com URL.

Cost: ~\$0.064/hour for the smallest config (1 vCPU, 2 GB) while running. Pause the service after demo day to stop billing.

[Tutorial: App Runner getting started \(AWS docs\)](#)

5.4 Amazon ECS on Fargate

Run Docker containers without managing servers. More flexible than App Runner — supports multiple containers, sidecars, custom networking, service discovery.

When to reach for it: you have two or more containers that need to talk to each other (frontend + backend + worker), or you need fine-grained VPC/IAM control. For one container, use App Runner instead.

Quick-start: ECS → Create cluster → Networking only (Fargate). Task definition with your image (from ECR). Service with desired count 1, attach to an ALB if public. There are too many decisions here for a numbered list — follow the linked guide step-by-step.

[Tutorial: Create a Fargate task and service \(AWS docs\)](#)

5.5 Amazon EKS (Kubernetes)

Managed Kubernetes. Pay \$0.10/hour for the control plane plus node costs.

When to reach for it: your team is already fluent in kubectl and your demo specifically needs Kubernetes features. For a 24-48 hour hackathon, this is almost always the wrong choice — use App Runner or ECS Fargate. Mentioned for completeness.

[Tutorial: EKS getting started \(AWS docs\)](#)

5.6 Amazon EC2

Virtual machines. You pick the OS, you install everything, you SSH in.

When to reach for it: you need a long-running process that doesn't fit serverless (a model server, a database you self-host, an experiment that needs GPU). For most hackathon use cases, Lambda or App Runner is faster to ship.

Console quick-start:

1. EC2 → Launch instance.
2. Name: team02-server. AMI: Amazon Linux 2023 (default) or Ubuntu 22.04.
3. Instance type: pick from the hackathon-approved list — **t3.micro, t3.small, t3.medium, t4g.micro, t4g.small, t4g.medium, m5.large, m6i.large, c5.large, c6i.large**. Default to t3.small unless you know you need more.
4. Key pair: create one if you don't have it. Download the .pem. Without this you cannot SSH.
5. Network: default VPC is fine. Security group: allow SSH from My IP, plus any app ports (e.g. 80, 443, 3000).
6. Storage: 20 GB gp3 is plenty. Launch instance.
7. Once running, copy the Public IPv4 and SSH: `ssh -i team02.pem ec2-user@<public-ip>`.

Cost watch: a t3.small is about \$0.021/hour (\$0.50/day). Stop the instance (not terminate) when you're done for the night — stopped instances don't bill compute.

[Tutorial: EC2 getting started \(AWS docs\)](#)

6. Messaging & Events

Three services that decouple producers from consumers. SQS for work queues (one consumer per message). SNS for fan-out (one message to many subscribers). EventBridge for routing events from AWS services and your own apps to multiple targets based on patterns.

6.1 Amazon SQS

Managed message queue. Producer drops a message, consumer pulls it, processes it, deletes it. If the consumer crashes, the message becomes visible again and another consumer picks it up.

When to reach for it: any work you want to do asynchronously — image processing, model inference jobs, scheduled batches. Pair with Lambda (configure SQS as a Lambda trigger) for fully serverless workers. 1M requests free monthly.

Console quick-start:

1. SQS → Create queue → Standard.
2. Name: team02-jobs.
3. Visibility timeout: set this to longer than your worker's expected processing time (default 30s is fine for fast jobs; bump to 5 minutes for ML inference).
4. Message retention: 4 days is the default. Leave as-is.
5. Create queue. Copy the queue URL.
6. To send: `aws sqs send-message --queue-url <url> --message-body '{"job":"resize-image","key":"foo.jpg"}'`
7. To consume from Lambda: open your Lambda → Configuration → Triggers → Add → SQS → pick the queue. Lambda will poll and invoke automatically.

[Tutorial: SQS getting started \(AWS docs\)](#)

6.2 Amazon SNS

Pub/sub. One topic, many subscribers (email, SMS, Lambda, SQS, HTTPS). Each subscriber gets every message published to the topic.

When to reach for it: send an alert email when something happens, notify multiple services about the same event, send SMS confirmations (SMS costs extra). 1M publishes free monthly.

Console quick-start:

1. SNS → Topics → Create topic → Standard.
2. Name: team02-alerts. Create.
3. Open the topic → Create subscription → Protocol Email → Endpoint your-email@example.com. Confirm via the email AWS sends you.
4. Publish a test message from the topic's Publish message button. You'll receive an email within a few seconds.

[Tutorial: SNS getting started \(AWS docs\)](#)

6.3 Amazon EventBridge

Event bus with pattern-based routing. Most useful feature for hackathons: cron-style schedules that trigger Lambda.

When to reach for it: you need 'run this Lambda every 5 minutes' or 'run this every day at 9am UTC'. Also: react to AWS service events (S3 object created, EC2 instance state change) without writing polling code.

Console quick-start (schedule a Lambda):

1. EventBridge → Schedules → Create schedule.
2. Name: team02-nightly-report. Schedule pattern: Recurring → Cron-based → 0 9 * * ? * (every day 9am UTC). Or Rate-based for simpler intervals.
3. Flexible time window: Off. Next.
4. Target: AWS Lambda → InvokeFunction → pick your function. Next.
5. Permissions: Create new role for this schedule (default). Next, Create schedule.

[Tutorial: EventBridge Scheduler getting started \(AWS docs\)](#)

7. Streaming — Amazon Kinesis

Real-time data streams. Use only if your idea genuinely streams data — clickstream analytics, IoT sensor data, log aggregation. For most hackathon ideas, SQS + Lambda is enough.

7.1 Kinesis Data Streams

An ordered, partitioned, durable stream. Producers write records; multiple consumers read them in parallel.

When to reach for it: many independent consumers need to process the same firehose of events (analytics + alerts + archive). Otherwise SQS is simpler and cheaper.

Console quick-start:

1. Kinesis → Data Streams → Create data stream.
2. Name: team02-events. Capacity mode: On-demand (no shard planning required, pay per use).
3. Create. The stream is ready in under a minute.
4. Produce records via SDK (boto3: `kinesis.put_record(StreamName='team02-events', Data=json.dumps(event), PartitionKey=user_id)`).
5. Consume via a Lambda trigger: Lambda → Triggers → Kinesis → pick the stream. Starting position: LATEST.

Cost watch: on-demand is roughly \$0.04/hour idle. Delete the stream after demo day.

[Tutorial: Kinesis Data Streams getting started \(AWS docs\)](#)

8. Databases

Two main choices. DynamoDB for key-value/document access patterns with predictable queries — serverless, scales to zero billing, ideal for hackathon prototypes. RDS/Aurora when you genuinely need SQL — joins, aggregations, ad-hoc reporting.

8.1 Amazon DynamoDB

Managed NoSQL key-value store. Single-digit millisecond reads, no servers to manage, pay-per-request billing means you owe nothing when idle.

When to reach for it: user profiles, session state, leaderboards, simple lookups by id. Default choice for hackathon backends. 25 GB storage + 25 RCU/WCU free monthly.

Console quick-start:

1. DynamoDB → Create table.
2. Table name: team02-users. Partition key: userId (String). Sort key: leave blank unless you need range queries.

3. Table settings: Customize → Capacity mode: On-demand (pay per request, no capacity planning).
4. Create table.
5. Use the Explore items view to add a few rows by hand. Or write from Lambda: `dynamodb.put_item(Table='team02-users', Item={'userId': {'S': 'u1'}, 'name': {'S': 'Asha'}})`.

Design tip: DynamoDB rewards thinking about your access patterns first. List the queries your app needs (get user by id, list users by signup_date, etc.) before designing keys.

[Tutorial: DynamoDB getting started \(AWS docs\)](#)

8.2 Amazon RDS / Aurora Serverless v2

Managed relational databases — PostgreSQL, MySQL, MariaDB. Aurora Serverless v2 auto-scales capacity and is the recommended flavor for variable hackathon workloads.

When to reach for it: your data is genuinely relational (joins across multiple tables), or you want to use Postgres extensions (pgvector for embeddings, PostGIS for geo).

Console quick-start:

1. RDS → Create database → Standard create.
2. Engine: Aurora (PostgreSQL Compatible). Templates: Dev/Test.
3. DB cluster identifier: team02-db. Master username: postgres. Master password: generate a strong one, save in your team password manager.
4. Instance configuration: Serverless v2. Capacity range: Min 0.5 ACU, Max 4 ACU. Allowed Instance Types : **db.t3.micro, db.t3.small, db.t3.medium, db.t4g.micro, db.t4g.small, db.t4g.medium, db.m5.large.**
5. Connectivity: VPC default, Public access Yes (hackathon convenience — never in production), VPC security group → Create new (allow inbound 5432 from your IP).
6. Create database. Provisioning takes ~10 minutes.
7. Once available, the writer endpoint shows in the Connectivity tab. Connect with `psql -h <writer-endpoint> -U postgres -d postgres`.

Cost watch: even at 0.5 ACU minimum, Aurora Serverless v2 runs ~\$0.06/hour while available. Delete the cluster after demo day. Take a final snapshot if you want to keep the data.

[Tutorial: Aurora Serverless v2 getting started \(AWS docs\)](#)

9. Storage & Analytics

If your idea involves crunching a dataset — uploaded CSVs, scraped data, logs — these services let you store, query, and search without spinning up infrastructure. S3 stores the raw data; Glue catalogues it; Athena queries it in place with SQL; Redshift is the heavy-duty warehouse for larger analytics; OpenSearch does full-text and vector search.

9.1 Amazon S3 (data storage)

Same service covered in Section 3 — here used as a data lake instead of a website host. Drop CSV, JSON, or Parquet files in a bucket; query them with Athena.

Console quick-start: S3 → Create bucket (e.g. team02-data) → Upload files. That's it for storage. Organize by prefix (raw/, processed/) — Glue and Athena treat each prefix as a partition.

[Tutorial: S3 user guide \(AWS docs\)](#)

9.2 AWS Glue Data Catalog

A metadata catalog. Run a Glue Crawler over an S3 prefix and it figures out the schema, registers a table, and lets Athena query it.

Console quick-start:

1. Glue → Crawlers → Create crawler.
2. Name: team02-crawler. Next.
3. Data source: S3, path s3://team02-data/raw/. Next.
4. IAM role: Create new role (AWSGlueServiceRole-team02). Next.
5. Output: Database → Add database team02_db. Schedule: On demand. Next, Create.
6. Run crawler. After 1-2 minutes the catalog has a table you can query in Athena.

Cost watch: Glue crawlers cost ~\$0.44/DPU-hour. A small crawl is pennies. Don't leave it on a schedule.

[Tutorial: Glue getting started \(AWS docs\)](#)

9.3 Amazon Athena

Serverless SQL over data sitting in S3. Pay \$5 per TB of data scanned — for hackathon datasets, this is usually fractions of a cent per query.

When to reach for it: you have CSV/Parquet in S3 and want to run SELECT statements without loading it into a database. Best paired with a Glue crawler.

Console quick-start:

1. Athena → Query editor.
2. First-time setup: Settings → Manage → set Query result location to `s3://team02-data/athena-results/`. Save.
3. Database dropdown: pick team02_db (created by your Glue crawler).
4. Write SQL: `SELECT * FROM your_table LIMIT 10;` → Run.
5. Results appear in the editor and are also written to your athena-results/ prefix.

[Tutorial: Athena getting started \(AWS docs\)](#)

9.4 Amazon Redshift

Petabyte-scale data warehouse. Use Redshift Serverless to avoid managing nodes. Allowed instance types : **dc2.large, ra3.xlplus**.

When to reach for it: hundreds of millions of rows that need fast analytics queries. For most hackathon datasets (under ~10 GB), Athena is simpler and cheaper. Use Redshift only if Athena's per-query latency hurts your demo.

[Tutorial: Redshift Serverless getting started \(AWS docs\)](#)

9.5 Amazon OpenSearch Service

Managed Elasticsearch. Full-text search, log analytics, and (with the k-NN plugin) vector search for semantic retrieval.

When to reach for it: your demo needs search-as-you-type, fuzzy matching, or you're building a RAG app and want to store embeddings alongside the source text. For pure RAG, also consider Bedrock Knowledge Bases — it manages this for you.

Console quick-start:

1. OpenSearch → Create domain → Easy create.
2. Name: team02-search. Engine: OpenSearch latest.
3. Network: Public access (hackathon only). Fine-grained access: master user → username/password.
4. Create. Provisioning takes ~15 minutes.
5. Once active, the dashboard URL gives you Kibana-like search and the endpoint URL is what your app posts to.

Cost watch: even the smallest OpenSearch domain runs ~\$0.10/hour. Delete the domain after the demo.

[Tutorial: OpenSearch getting started \(AWS docs\)](#)

10. Orchestration

Once you have multiple Lambdas or Glue jobs, you'll want to coordinate them. Step Functions for short workflows expressed as a state machine. MWAA (managed Airflow) if your team already runs Airflow DAGs — otherwise skip, it's heavy.

10.1 AWS Step Functions

Visual workflow engine. Chain Lambda invocations, retries, parallel branches, and waits into a single state machine.

When to reach for it: a workflow that's more than 'one Lambda calls another'. Image upload → resize → analyze → notify is a classic three-step Step Function.

Console quick-start:

1. Step Functions → Create state machine.
2. Use the visual Workflow Studio. Drag a Lambda Invoke action onto the canvas, pick your function.
3. Add more states by dragging — Choice for branching, Parallel for fan-out, Wait for delays.

4. Configuration: Type Standard (cheap, long-lived) or Express (cheaper for high-volume short workflows). For hackathons, Standard.
5. Create. Start an execution with sample input from the console to see the run light up the diagram in real time.

[Tutorial: Step Functions getting started \(AWS docs\)](#)

10.2 Amazon MWAA (Managed Airflow)

Managed Apache Airflow. Same DAGs your team already writes, AWS runs them.

When to reach for it: only if your team has existing Airflow DAGs you want to lift-and-shift. For new hackathon work, Step Functions is faster to set up and cheaper.

[Tutorial: MWAA getting started \(AWS docs\)](#)

11. Operations & Security

Once anything is running in your account, three questions matter: is it healthy, who can touch it, and how much is it spending. The services in this section answer those.

11.1 Amazon CloudWatch

Metrics, logs, and alarms — the observability backbone for everything in your account. Lambda logs land here automatically. So do API Gateway access logs, ECS task logs, and most other services with a 'send logs' checkbox.

When to reach for it: anytime something is broken. The Logs Insights query editor is your friend.

Console quick-start:

1. CloudWatch → Logs → Log groups. Find /aws/lambda/<your-function> after invoking your Lambda once. Click in.
2. Click a recent log stream. Each invocation's START / END / REPORT lines plus any console.log output is here.

3. Logs → Logs Insights for cross-stream queries: pick the log group(s), paste a query like fields @timestamp, @message | filter @message like /ERROR/ | sort @timestamp desc | limit 50.
4. Alarms → Create alarm. Pick a metric (e.g. Lambda Errors > 5 in 5 minutes), set an SNS topic to email you, save.

[Tutorial: CloudWatch Logs Insights query syntax](#)

11.2 AWS X-Ray

Distributed tracing. Shows you, request by request, where time is being spent across Lambda → DynamoDB → external API calls. Worth the 10 minutes if you have more than two services in a chain.

When to reach for it: a request feels slow and you can't tell which hop is responsible.

Console quick-start:

1. On the Lambda function → Configuration → Monitoring and operations tools → Edit → toggle Active tracing on.
2. Make a few requests through your app.
3. X-Ray → Traces. Pick a recent trace. The waterfall shows every downstream call with timing.
4. X-Ray → Service map for a graph view of how your services call each other.

[Tutorial: Instrument a Lambda function with X-Ray](#)

11.3 AWS Secrets Manager

Stores API keys, DB passwords, and OAuth tokens encrypted at rest, with rotation hooks. Your code calls GetSecretValue at startup instead of reading from .env.

When to reach for it: any time you would otherwise commit a credential to Git. Especially: third-party API keys and DB passwords.

Console quick-start:

1. Secrets Manager → Store a new secret → Other type of secret.
2. Add key-value pairs (e.g. OPENAI_API_KEY = sk-...). Encrypt with default KMS key.
3. Name it (e.g. team02/openai). Skip rotation for hackathon use. Store.

4. In your Lambda's execution role, attach a policy allowing `secretsmanager:GetSecretValue` on that secret's ARN.
5. In code: `const {SecretsManagerClient, GetSecretValueCommand} = require('@aws-sdk/client-secrets-manager')`. Fetch on cold start, cache in module scope.

[Tutorial: Retrieving a secret from your Lambda function](#)

Cost note: \$0.40 per secret per month + \$0.05 per 10,000 API calls. Cache values in Lambda module scope to keep calls down.

11.4 AWS KMS (Key Management Service)

Encryption-key management. Most AWS services already use KMS under the hood with AWS-managed keys (S3 SSE-S3, DynamoDB encryption at rest, Secrets Manager). You rarely need to create your own key for hackathon work.

When to reach for it: you have a regulatory or audit reason to control your own key. Otherwise, use the AWS-managed keys that ship with each service.

11.5 AWS Systems Manager — Parameter Store

Hierarchical key-value store for configuration. Free for standard parameters, cheaper than Secrets Manager when you don't need rotation. Great for environment-specific config (feature flags, public URLs, model names).

When to reach for it: non-secret config that varies by environment. For real secrets, use Secrets Manager.

Console quick-start:

1. Systems Manager → Parameter Store → Create parameter.
2. Name with a path prefix, e.g. `/team02/api/base-url`. Type: String (or `SecureString` for sensitive values, which uses KMS).
3. In Lambda IAM role, add `ssm:GetParameter` and `ssm:GetParameters` on the path's ARN.
4. In code: `SSMClient.send(new GetParameterCommand({Name: '/team02/api/base-url'}))`.

[Tutorial: SSM Parameter Store walkthrough](#)

11.6 AWS WAF

Web application firewall in front of CloudFront, API Gateway, or an Application Load Balancer. Blocks common attack patterns (SQL injection, XSS) and lets you rate-limit by IP.

When to reach for it: your demo will be live on the public internet and you want a basic safety net. The AWS-managed Core rule set covers the obvious threats.

Console quick-start:

1. WAF & Shield → Web ACLs → Create web ACL. Region: us-east-1 (or CloudFront for global).
2. Associated resources: pick your API Gateway / CloudFront distribution / ALB.
3. Add rules → Add managed rule groups → tick `AWSManagedRulesCommonRuleSet` and `AWSManagedRulesKnownBadInputsRuleSet`.
4. Default action: Allow. Save.

[Tutorial: WAF quickstart with managed rules](#)

Cost note: ~\$5 per web ACL per month + per-request charges. Leave WAF off unless you specifically want it.

11.7 AWS Shield

DDoS protection. The Standard tier is automatically on for every CloudFront, ALB, and Route 53 resource at no charge — no action required. Shield Advanced is enterprise-grade and costs \$3,000/month; do not enable it.

When to reach for it: you don't. Standard is already protecting your CloudFront/API Gateway.

12. AI & Machine Learning

AI/ML is where many AI4Good projects spend most of their time. AWS gives you two starting points: Bedrock if you want to call a foundation model with an API key, or SageMaker if you want to train, fine-tune, or host your own model.

12.1 Amazon Bedrock

Foundation models as an API: Anthropic Claude, Meta Llama, Mistral, Amazon Titan, Cohere, Stability AI. One IAM role, one SDK call, no GPU management. Includes Knowledge Bases (RAG), Agents (tool-using LLMs), and Guardrails (safety filters).

When to reach for it: any time your idea has 'use an LLM' in it. This is the default for AI4Good projects unless you have a specific reason to host your own model.

Console quick-start:

1. Bedrock → Model access (left nav) → Modify model access.
2. Tick the models you want. Anthropic Claude 3.5 Sonnet and Claude 3 Haiku are good defaults — request access (instant approval for most).
3. Test in the Playground: Bedrock → Chat / Test → pick the model → type a prompt.
4. From code: use `@aws-sdk/client-bedrock-runtime`, `InvokeModel` command. The model ID is `anthropic.claude-3-5-sonnet-20240620-v1:0` (or the latest version listed).
5. For RAG: Bedrock → Knowledge Bases → Create. Pick S3 as the data source, OpenSearch Serverless or Aurora pgvector as the vector store.

[Tutorial: Amazon Bedrock for Beginners — From First Prompt to AI Agent \(YouTube\)](#)

[Tutorial: Bedrock Knowledge Bases, Guardrails, and Security \(YouTube\)](#)

Cost note: pay per 1,000 input/output tokens. Claude 3.5 Sonnet ~\$3 input / \$15 output per million tokens. Haiku is ~10x cheaper and fast enough for most prototypes — start there.

12.2 Amazon SageMaker

Managed platform for the full ML lifecycle: notebooks, training jobs, model hosting, batch inference. Use SageMaker Studio for an IDE-style experience, or just SageMaker Notebooks for a Jupyter environment.

When to reach for it: you are training or fine-tuning a model, or you need a notebook with a GPU. Skip this if your project only needs to call an LLM (use Bedrock).

Console quick-start:

1. SageMaker → Studio → Set up for single user (Quick setup). This provisions a domain and a default user.
2. Once the domain is Ready, click Open Studio. The browser IDE opens.

3. File → New → Notebook. Pick a kernel image (Data Science 3.0) and allowed instance type : **ml.t3.medium, ml.t3.large, ml.m5.large, ml.m5.xlarge.**

4. Stop the instance when you're done — running notebooks are billed by the hour.

[Tutorial: Get started with SageMaker Studio](#)

Cost note: ml.t3.medium is ~\$0.05/hour. GPU instances (ml.g4dn.xlarge) are ~\$0.50/hour. Stop notebooks when not in use — you pay while they are running.

12.3 Amazon Lex

Conversational interface builder — same engine that powers Alexa. Intent recognition, slot filling, multi-turn dialog. Integrates with Lambda for fulfillment.

When to reach for it: you are building a chatbot with structured intents (book a slot, check status, file a request) rather than open-ended LLM chat. For free-form chat, Bedrock is the better starting point.

[Tutorial: Build a Lex V2 bot from the console](#)

13. Communications & CI/CD

13.1 Amazon SES (Simple Email Service)

Transactional email — verification emails, password resets, notifications. Sandbox mode by default (can only send to verified addresses); production mode requires a one-form review.

When to reach for it: your demo needs to send email. For SMS, use SNS instead.

Console quick-start:

1. SES → Verified identities → Create identity → Email address. Enter the sender address. AWS emails a verification link — click it.
2. Verify each recipient address the same way (or request production access if your demo emails real users).
3. Send a test email from SES → Verified identities → your address → Send test email.

4. From code: use @aws-sdk/client-sesv2, SendEmailCommand. Source = verified sender, Destination = verified recipient (until production).

[Tutorial: SES getting started](#)

13.2 CodePipeline + CodeBuild + CodeCommit

AWS-native CI/CD. CodeCommit hosts a Git repo, CodeBuild runs build/test/package, CodePipeline orchestrates the stages. If your team is already on GitHub, prefer Amplify Hosting (Section 3.3) for frontend or a simple GitHub Actions workflow for backend — Code* is heavier than most hackathon projects need.

When to reach for it: you specifically want everything inside AWS (CodeCommit) or you have a containerized backend deploying to ECS/EKS that needs an automated build → push to ECR → update service pipeline.

[Tutorial: CodePipeline simple pipeline walkthrough](#)

14. Cost guardrails & support

Hackathon accounts have a real credit cap. Setting up two guardrails in the first 10 minutes saves the team from a surprise on day two.

14.1 AWS Budgets

Set a dollar threshold and get an email when forecasted spend will cross it. Free for the first two budgets per account.

Console quick-start:

1. Billing and Cost Management → Budgets → Create budget. Use a template → Monthly cost budget.
2. Set the budget amount to whatever cap your team has been told (e.g. \$50 for the event).
3. Email recipients: add every team member.
4. Create. You'll get an alert at 80% actual and 100% forecasted.

14.2 AWS Cost Explorer

Spend dashboard, grouped by service or tag. Updates daily. Use it on day two to see what's actually been costing money.

Console quick-start:

1. Billing and Cost Management → Cost Explorer. (First open takes a few minutes to enable.)
2. Group by → Service. The chart shows the last 30 days of spend by service.
3. If one service is unexpectedly high, jump to its console and check what is running. Common culprits: idle EC2 instances, RDS clusters left on, NAT Gateways.

14.3 End-of-hackathon cleanup

Before you walk away, kill the long-running stuff. The credit doesn't reset.

1. EC2 → Instances → terminate anything you do not need.
2. RDS → Databases → delete (uncheck final snapshot to skip the storage charge).
3. EKS / ECS → delete clusters and services.
4. NAT Gateway: if you created one, delete it (this is the most common forgotten-resource that burns credit).
5. S3 buckets with large data: empty and delete.
6. Cost Explorer: confirm yesterday's spend dropped to near-zero before logging out.

14.4 Getting help

If something is blocking your team and you cannot find a path in the docs, the Blend team running the event is on-site. Flag a Blender — that's the fastest path to unblock. The AWS documentation is the second-fastest path; AI assistants are a fine third when you know enough to phrase the question.

Good luck!